

kaiten-zushi

Wiktor Sasnal

Zaawansowana symulacja restauracji sushi oparta na architekturze wieloprocessowej oraz mechanizmach komunikacji międzyprocessowej w systemie Linux.

Spis treści

- [AUTOR](#)
- [TEMAT PROJEKTU](#)
- [INFORMACJE OGÓLNE](#)
- [TECHNOLOGIE](#)
- [ARCHITEKTURA IPC](#)
- [TESTY](#)
- [KONFIGURACJA](#)

AUTOR

Wiktor Sasnal

TEMAT PROJEKTU

Temat 1 - Restauracja „kaiten zushi”.

INFORMACJE OGÓLNE

1. Założenia projektowe i opis kodu

Celem projektu było stworzenie symulacji restauracji: Kaiten-zushi. Kod został podzielony na moduły: **main** (zarządca), **klient** (logika gości), **kucharz**, **obsługa** i **kierownik**.

2. Elementy specjalne

- **VIP i Napiwki:** Implementacja napiwków dla vipów jako dodatkowy przepływ gotówki nieuwzględniony w poleceniu.
-

TECHNOLOGIE



ARCHITEKTURA IPC

A. Tworzenie i obsługa plików

- [Zapis logów do pliku: common.h](#)

B. Tworzenie procesów

- [Tworzenie procesu klienta: main.c](#)
- [Uruchamianie programu np.klienta: main.c](#)
- [Kończenie procesu: klient.c](#)
- [Oczekiwanie na procesy potomne zombie: main.c](#)

C. Tworzenie i obsługa wątków

- [Tworzenie wątków osób w grupie: klient.c](#)
- [Oczekiwanie na wątki: klient.c](#)
- [Synchronizacja wątków: klient.c](#)

D. Obsługa sygnałów

- [Rejestracja obsługi SIGINT/SIGALRM\(signal/sigaction\): main.c](#)
- [Wysyłanie sygnałów do pracowników\(kill\): main.c](#)

E. Synchronizacja procesów

- [Inicjalizacja zestawu semaforów \(semget/semctl\): main.c](#)
- [Operacje na semaforach \(semop - funkcja pomocnicza\): common.h](#)

F. Pamięć dzielona

- [Utworzenie segmentu pamięci \(shmget\): main.c](#)
- [Dołączenie pamięci do przestrzeni adresowej \(shmat\): main.c](#)
- [Usunięcie segmentu \(shmctl IPC_RMID\): main.c](#)

G. Kolejki komunikatów (Message Queues)

- [Utworzenie kolejki \(msgget\): main.c](#)
- [Wysłanie zamówienia specjalnego \(msgsnd IPC_NOWAIT\): klient.c](#)
- [Odbiór zamówienia przez kucharza \(msgrcv\): kucharz.c](#)

Implementacja struktur **IPC** dla zasady zadania projektowego

1. Pamięć Współdzielona

Wspólny obszar pamięci przechowuje stan świata dostępny dla wszystkich procesów:

- **BeltSlot belt[P]**: Taśma z posiłkami (dostęp chroniony mutexem).
- **table_capacity / current_occupancy**: Tablice monitorujące obłożenie każdego stolika/miejsca.
- **stats_***: Globalne statystyki finansowe (sprzedaż, koszty produkcji, napiwki).
- **Flagi sterujące**: **open** (czy lokal otwarty), **emergency_exit** (ewakuacja).

Link do kodu (implementacja): [Kliknij tutaj, aby zobaczyć kod](#)

2. Semafor

System wykorzystuje zestaw **8 semaforów** do sterowania dostępem i synchronizacji:

ID	Rola	Opis działania
0	Mutex Taśmy	Binarny (0/1). Blokuje dostęp do edycji taśmy podczas nakładania/zdejmowania dań.
1	Sygnal Kucharza	Kucharz oczekuje na tym semaforze (wartość 0). Klient podbija go (+1), by zlecić zamówienie.
2	Licznik 2-os	Sem. licznikowy dla stolików 2-osobowych.
3	Licznik 3-os	Sem. licznikowy dla stolików 3-osobowych.
4	Licznik 4-os	Sem. licznikowy dla stolików 4-osobowych.
5	Mutex Kasjera	Binarny (0/1). Zapewnia atomowość operacji dodawania utargu do statystyk globalnych.
6	Licznik Lady	Sem. licznikowy. Liczba wolnych miejsc przy barze (Lada).
7	Licznik 1-os	Sem. licznikowy dla stolików 1-osobowych.

Link do kodu (implementacja): [Kliknij tutaj, aby zobaczyć kod](#)

3. Kolejki Komunikatów

- **Cel:** Obsługa asynchronicznych Zamówień Specjalnych.
- **Działanie:** Klient wysyła strukturę `SpecialOrder`.

Link do kodu (implementacja): [Kliknij tutaj, aby zobaczyć kod](#)

4. Wątki

- **Kontekst:** Działają wewnątrz procesu `./klient`.
- **Rola:** Symulują poszczególne osoby w grupie siedzące przy jednym stoliku.
- **Synchronizacja:** `pthread_mutex_t` `group_lock` chroni wspólny rachunek grupy oraz licznik zjedzonych posiłków.

Link do kodu (implementacja): [Kliknij tutaj, aby zobaczyć kod](#)

TESTY

Poniżej przedstawiono zestawienie testów weryfikujących kluczowe funkcjonalności systemu. Każdy test opatrzony jest dowodem działania.

T1: Weryfikacja Opieki nad Dziećmi (Odmowa Wstępu)

- **Cel:** Sprawdzenie, czy system blokuje wejście grupie, która nie spełnia wymogu opieki (minimum 1 dorosły na każde rozpoczęte 3 dzieci).
- **Scenariusz:** Uruchomienie grupy , w której losowanie przydzieliło same dzieci (wiek < 10 lat).

- **Oczekiwany Rezultat:**

1. Program wypisuje czerwony komunikat: `[System] ODMOWA WSTĘPU! Za mało opiekunów.`
2. W komunikacie widać szczegóły grupy (wiek uczestników).
3. Proces klienta kończy się natychmiast (`exit(0)`) i nie zajmuje zasobów.

Dowód działania:

```
[System] Grupa 2 os. Wiek: 10 3
[System] ODMOWA WSTĘPU! Za mało opiekunów.
```

T2: Logika Współdzielenia Stolików

- **Cel:** Sprawdzenie, czy semaforey i algorytm wyszukiwania pozwalają na dosiadanie się grup do częściowo zajętych stolików.
- **Scenariusz:**
 1. Do stolika 4-osobowego (np. o indeksie 16) siada pierwsza grupa 2-osobowa.
 2. Wchodzi kolejna grupa 2-osobowa, która wylosuje ten sam typ stolika/strefę.
- **Oczekiwany Rezultat:**
 1. Druga grupa otrzymuje ten sam numer stolika co pierwsza (np. nr 16).
 2. Pojawia się komunikat: `DOSIADA SIĘ do stolika nr 16.`
 3. Obie grupy jedzą równolegle, a zajętość stolika w pamięci współdzielonej wynosi 4/4.

Dowód działania:

```
=====
[Grupa 2349457] ZAJMUJE PUSTY STOLIK nr 21 (Pojemnosc: 4) (LUZNY STOLIK)
>> Wielkosc: 2 os. CEL DO ZJEDZENIA: 3 DAN.
=====
[ZEGAR] Godzina: 10:06
[System] Grupa 4 os. Wiek: 27 60 48 27

=====
[VIP 2349551] DOSIADA SIĘ do stolika nr 21 (Pojemnosc: 4)
>> Wielkosc: 2 os. CEL DO ZJEDZENIA: 12 DAN.
=====
[Klient 2349551-1] >> ZAMAWIA DANIE SPECJALNE (Koszt: 50 zl)
[ZEGAR] Godzina: 10:39
```

T3: Weryfikacja Celu Konsumpcji

- **Cel:** Sprawdzenie, czy klient poprawnie realizuje cykl życia: jedzenie -> osiągnięcie celu -> zwolnienie stolika.
- **Scenariusz:** Klient otrzymuje losowy cel zjedzenia dań (zmienna `target_to_eat`, np. 4 dań).
- **Oczekiwany Rezultat:**
 1. W logu startowym widać: `>> Wielkosc: X os. CEL DO ZJEDZENIA: 5 DAN.`
 2. Klient zjada dokładnie tyle dań (lub więcej, jeśli kończył równocześnie z innymi).
 3. W komunikacie końcowym (po wyjściu) widnieje status `Najedzeni` oraz poprawna kwota rachunku.

Dowód działania:

```
=====
[Grupa 2366594] ZAJMUJE MIEJSCE PRZY LADZIE (Nr 0)
>> Wielkosc: 1 os. CEL DO ZJEDZENIA: 8 DAN.
=====
```

```
[ZEGAR] Godzina: 10:24
```

```
[Grupa 2366594] KONIEC (Status: Najedzeni). Rachunek: 120 zl.
SZCZEGOLY KONSUMPCJI:
- 2 x Danie Typ 1 (10 zl)
- 4 x Danie Typ 2 (15 zl)
- 2 x Danie Typ 3 (20 zl)
=====
```

T4: Poprawność Raportu Finansowego

- **Cel:** Weryfikacja, czy proces **Main** poprawnie sumuje przychody ze wszystkich źródeł (sprzedaż standardowa, specjalna, napiwki).
- **Scenariusz:** Symulacja z udziałem klientów VIP (zamawiających dania specjalne i dających napiwki) oraz zwykłych klientów.
- **Oczekiwany Rezultat:**
 1. W raporcie końcowym wyświetlanym po zamknięciu lokalu sumy się zgadzają.
 2. Równanie: **Sprzedaż Dań Podstawowych + Sprzedaż Dań Specjalnych + Napiwki = CAŁKOWITY PRZYCHÓD.**

Dowód działania:

```
[Obsluga] Koniec pracy.
[Kierownik] Main zamknal lokal (open=false). Koncze prace.

=====
                RAPORT KONCOWY DNIA
=====

[Kucharz] STATYSTYKA PRODUKCJI:
- Danie Typ 1: 33 szt. (Koszt: 330 zl)
- Danie Typ 2: 30 szt. (Koszt: 450 zl)
- Danie Typ 3: 34 szt. (Koszt: 680 zl)
- Danie Typ 4: 12 szt. (Koszt: 480 zl)
- Danie Typ 5: 10 szt. (Koszt: 500 zl)
- Danie Typ 6: 5 szt. (Koszt: 300 zl)
LACZNY KOSZT PRODUKCJI: 2740 zl

-----

[Kasjer] STATYSTYKA SPRZEDAZY:
- Sprzedaz Dan Podstawowych: 1145 zl
- Sprzedaz Dan Specjalnych : 1280 zl
- Napiwki (VIP)           : 373 zl
CALKOWITY PRZYCHOD: 2798 zl

-----

[Sprzatacz] STRATY (Jedzenie na tasmie):
- Slot 0: 20 zl
- Slot 1: 15 zl
- Slot 2: 15 zl
- Slot 3: 15 zl
- Slot 4: 10 zl
- Slot 5: 20 zl
- Slot 6: 10 zl
- Slot 7: 10 zl
- Slot 8: 15 zl
- Slot 9: 20 zl
- Slot 10: 20 zl
- Slot 11: 10 zl
- Slot 12: 10 zl
- Slot 13: 20 zl
- Slot 14: 20 zl
- Slot 15: 10 zl
- Slot 16: 10 zl
- Slot 17: 15 zl
- Slot 18: 20 zl
- Slot 19: 10 zl
- Slot 20: 20 zl
ZMARNOWANE JEDZENIE: 315 zl

=====

[Main] Sprzatanie procesow pracownikow...
[Main] Zasoby IPC usuniete.
[Main] System zamkniete poprawnie.
```

KONFIGURACJA

To run this project:

1. Kompilacja modułów

Projekt można skompilować ręcznie przy użyciu poniższych komend:

```
# Sklonuj repozytorium
git clone https://github.com/wiksas/kaiten-zushi.git
```

```
cd kaiten-zushi

# Kompilacja ról systemowych
gcc kucharz.c -o kucharz
gcc obsluga.c -o obsluga
gcc kierownik.c -o kierownik

# Klient wymaga biblioteki pthread
gcc klient.c -o klient -lpthread

# Główny program
gcc main.c -o main
```

Uruchom symulację: `./main`.

Krok 2: Wysyłanie poleceń

W nowym oknie terminala wyślij odpowiedni sygnał:

Akcja	Sygnał	Komenda terminala
Przyspieszenie	SIGUSR1	kill -USR1 <PID_KIEROWNIKA>
Spowolnienie	SIGUSR2	kill -USR2 <PID_KIEROWNIKA>
Ewakuacja / Wyjście	SIGALRM	kill -ALRM <PID_KIEROWNIKA>

[Portfolio](#)